Dijkstra Is NOT Greedy: A Global-to-Local Proof of Correctness

Shengbao Wang
Hangzhou Normal University

Abstract

Dijkstra's algorithm is traditionally classified as a greedy algorithm. This paper argues that this classification misrepresents the mechanism by which the algorithm determines shortest paths. We present a direct global-to-local proof of correctness. At each iteration, the algorithm identifies a shortest path over the entire set of paths whose endpoints remain unsolved. This global minimum is necessarily a shortest path to its own endpoint. More fundamentally, the algorithm uses already determined shortest paths to establish one additional shortest path at each iteration. We identify this progression as dynamic programming. A three-step exclusion proof shows how the full path space is reduced to frontier candidates, how relaxation removes inferior alternatives, and why minimum extraction yields the global minimum. The choice is thus determined by a global comparison, not by an optimistic local greedy choice. We conclude that the algorithm is more accurately characterized by dynamic programming and deterministic path exclusion than by the traditional greedy classification.

1 Introduction

Since Dijkstra introduced it in 1959 [1], this elegant shortest-path algorithm has become one of the best-known algorithms in computer science. It has important practical applications, including route planning in map navigation and routing in communication networks [2]. It has also long been regarded as a standard greedy algorithm. This classification is prominent in classical textbooks [3, 4, 5]. Kleinberg and Tardos describe greedy decisions as being made *myopically*, with each step optimizing a local criterion. Cormen et al. characterize Dijkstra as greedy because it selects the unsettled vertex with the smallest tentative distance. Their correctness proof then establishes the finality of the selected vertex's distance. Sedgewick and Wayne present the same minimum-distance and relaxation operations, while Princeton course materials likewise place shortest paths among greedy algorithms [2, 6]. Roughgarden offers a particularly clear example: a minimum boundary candidate is selected, and its optimality for the chosen endpoint is subsequently proved [7].

These proofs are rigorous. They also employ properties of paths that cross the boundary of the settled set. Their principal conclusion, however, concerns the shortest-path distance of the *selected endpoint*. The choice is stated before its correctness is established. This order supports the customary picture of a local greedy decision followed by a proof of its safety. It does not expose the stronger fact that the selected candidate is the minimum over *all paths to currently unsolved destinations*.

The dynamic-programming character of shortest-path algorithms has also been studied. Bellman's principle of optimality and Lawler's treatment provide important foundations [8, 9]. Sniedovich analyzes Dijkstra as a dynamic-programming procedure, while Misra gives a fixed-point interpretation [10, 11]. Lew examines the relationship between greedy algorithms and dynamic programming, but still describes Dijkstra as greedy [12]. These studies reveal impor-

tant recursive features. They do not make the global minimization over all unsolved-destination paths the central object of the correctness argument.

This paper takes that global path space as its starting point. We prove that every iteration identifies its absolute shortest path. The shortest path to the selected endpoint then follows directly from this global result. To establish this claim, we give a three-step exclusion proof. The first step restricts the search to the frontier. The second uses optimal substructure and relaxation to retain the best candidate for each frontier vertex. The third determines the minimum over the remaining candidates. These reductions correspond to the operations of Dijkstra's algorithm itself.

The resulting interpretation reverses the familiar explanatory order. The algorithm does not reach a global conclusion by trusting a locally attractive choice. Instead, deterministic exclusions establish the global minimum before its endpoint is declared solved. Each iteration uses previously established shortest paths to obtain a new one. We therefore argue that Dijkstra's algorithm is fundamentally a dynamic-programming algorithm, and that its customary classification as greedy is incorrect. This claim concerns the mathematical basis of its minimum selection, not the validity of earlier correctness proofs.

2 Preliminaries and the Search-Space Partition

Let $G = (V, E, w)$ be a finite directed weighted graph with source $s \in V$. To make the exclusion argument especially transparent, we assume strictly positive edge weights throughout:

$$w(e) > 0 \qquad (e \in E).$$

For a path P, let $w(P)$ be the sum of its edge weights. Write $\delta(s, v)$ for the shortest-path distance to a vertex v reachable from s. Vertices not reachable from s are not included in the finite-distance minimizations below; if no unsolved vertex is reachable, the procedure terminates.

2.1 A motivating first step

The source s itself may be regarded as the first solved vertex, at distance 0. The next vertex to be solved is particularly easy to identify: it is an endpoint of a minimum-weight outgoing edge from s. Indeed, putting

$$m = \min_{(s,v)\in E} w(s, v),$$

any such one-edge path has length m. Every path from s with two or more edges must start with an outgoing edge whose length is at least m and must then add at least one positive-weight edge. Thus every multi-edge path is strictly longer than m. The minimum-weight one-edge path is the absolute shortest among all nonempty paths starting at s.

The underlying observation is elementary: *adding an edge to a path necessarily makes it longer.* It also prompts the question driving this paper. Does every subsequent iteration likewise identify an absolute shortest path from a global set of paths that are still relevant? Our proof answers yes.

2.2 Dijkstra's original three-way partition

Dijkstra already divided vertices into three classes in Problem 2 of his original paper [1]. Set A contains vertices with known shortest paths. Set B contains the remaining vertices directly

connected to A; the next vertex is chosen from this set. Set C contains all other vertices. We use different symbols but retain this three-region structure.

At iteration t, let $\mathcal{S}_t$ be the *solved set*, for which

$$d(x) = \delta(s, x) \qquad (x \in \mathcal{S}_t),$$

and let $\mathcal{U}_t = V \setminus \mathcal{S}_t$ be the *unsolved set*. The initially solved set is $\{s\}$, with $d(s) = 0$. Within $\mathcal{U}_t$, define

$$\mathcal{F}_t = \{f \in \mathcal{U}_t : \exists x \in \mathcal{S}_t \text{ such that } (x, f) \in E\}$$

as the *frontier* and $\mathcal{I}_t = \mathcal{U}_t \setminus \mathcal{F}_t$ as the *invisible set*. These are disjoint partitions:

$$V = \mathcal{S}_t \dot\cup \mathcal{U}_t, \qquad \mathcal{U}_t = \mathcal{F}_t \dot\cup \mathcal{I}_t.$$

Here "invisible" means only that the vertex is not directly exposed by an outgoing edge from the current solved set. Figure 1 illustrates Dijkstra's original three-region partition in our notation. For directed graphs, the frontier uses edges directed *out of* the solved region.

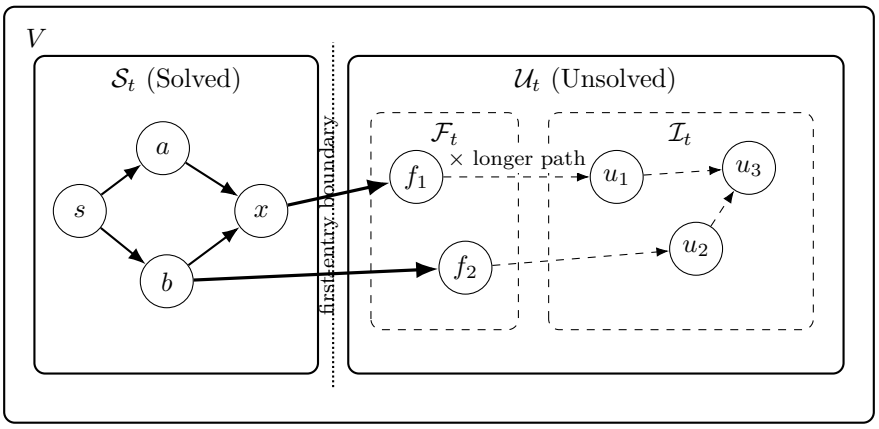


Figure 1: Dijkstra's three-way partition at iteration t, in our notation. Every path from s to an unsolved vertex first enters the frontier $\mathcal{F}_t$. With positive edge weights, any further extension is longer than this first-entry prefix.

2.3 The first-entry property

The partition has a simple but decisive consequence.

Lemma 1 (First-entry lemma)**.** *Let $P = (v_0 = s, v_1, \ldots, v_k = u)$ be any path ending at $u \in \mathcal{U}_t$. If v_j is the first vertex of P in $\mathcal{U}_t$, then $v_{j-1} \in \mathcal{S}_t$ and $v_j \in \mathcal{F}_t$.*

Proof. By the choice of j, all preceding vertices of P, in particular v_{j-1}, belong to $\mathcal{S}_t$. Since $(v_{j-1}, v_j) \in E$ and $v_j \in \mathcal{U}_t$, the definition of $\mathcal{F}_t$ gives $v_j \in \mathcal{F}_t$. □

Thus every path from the source into the unsolved region first reaches a frontier vertex. Even a path ending deep in $\mathcal{I}_t$ cannot bypass this boundary. This fact will justify the first exclusion in Section 4.

3 The Global-to-Local Principle and Path Exclusion

3.1 The global path space

For each $u \in \mathcal{U}_t$, let $\mathcal{P}(s,u)$ be the set of paths from s to u. Define the global search space at iteration t by

$$\mathcal{P}(\mathcal{U}_t) = \bigcup_{u \in \mathcal{U}_t} \mathcal{P}(s,u).$$

The objective of the iteration is to identify

$$P_t^* \in \arg\min_{P \in \mathcal{P}(\mathcal{U}_t)} w(P).$$

The minimum is taken over *all paths ending at unsolved vertices*, not over a collection of shortest paths that are assumed to have already been found.

3.2 From a global minimum to a local shortest path

Suppose P_t^* ends at u^*. Since

$$\mathcal{P}(s,u^*) \subseteq \mathcal{P}(\mathcal{U}_t),$$

every path Q from s to u^* satisfies

$$w(P_t^*) \leq w(Q).$$

Thus a path that minimizes the global search space necessarily minimizes the path set for its own endpoint. *One global shortest path therefore settles one additional vertex.* The remaining task is to show how Dijkstra determines this global minimum without enumerating all paths.

3.3 The exclusion principle

To find the shortest path, one may exclude any path that is strictly longer than another admissible candidate. Indeed, if P_1 is admissible and $w(P_1) < w(P_2)$, then P_2 cannot attain the current minimum. If several candidates tie, retaining one is sufficient to determine a shortest path.

This exclusion refers to the *current iteration.* A path extended beyond the first unsolved vertex may be irrelevant to the present global minimum, but its later edges can become relevant after that vertex is settled. The three-step proof below uses this distinction to reduce the current search space without losing any future shortest-path solution.

4 A Three-Step Exclusion Proof of Correctness

Fix iteration t with a nonempty reachable frontier. Distances $d(x) = \delta(s,x)$ for all $x \in \mathcal{S}_t$ are already finalized. We prove that Dijkstra's next extraction realizes the global minimum in $\mathcal{P}(\mathcal{U}_t)$.

4.1 Step 1: Frontier exclusion

Take an arbitrary path

$$P = (s = v_0, v_1, \ldots, v_k = u) \in \mathcal{P}(\mathcal{U}_t).$$

Let v_j be its first unsolved vertex. By Lemma 1, $v_j \in \mathcal{F}_t$. Its prefix $P^{\text{first}} = (v_0, \ldots, v_j)$ also belongs to $\mathcal{P}(\mathcal{U}_t)$. If P continues past v_j, it must acquire at least one more positive-weight edge, so

$$w(P^{\text{first}}) < w(P).$$

Thus no path containing more than one unsolved vertex can be the current global minimum: a shorter path already ends at its first frontier vertex. What remains relevant at this iteration is the form

$$s \rightsquigarrow x \to f, \qquad x \in \mathcal{S}_t, \quad f \in \mathcal{F}_t,$$

with every internal vertex in the solved set and exactly one unsolved vertex, its endpoint f. This is the mathematical reason why it suffices to consider frontier vertices rather than paths going further into the unsolved region.

4.2 Step 2: Relaxation exclusion

Fix $f \in \mathcal{F}_t$. Every candidate surviving Step 1 is of the form

$$P = Q(s, x) \circ (x, f), \qquad x \in \mathcal{S}_t.$$

First consider the prefix to x. Since x has already been solved, an arbitrary prefix $Q(s, x)$ cannot improve on $d(x) = \delta(s, x)$. If it is strictly longer, replacing it by a finalized shortest prefix produces a strictly shorter candidate to f. We therefore need only the candidate value

$$d(x) + w(x, f)$$

for each boundary edge (x, f). This is the optimal-substructure reason why relaxation extends finalized shortest distances, rather than enumerating earlier paths.

Several solved predecessors may lead to the same frontier vertex f. The shortest candidate among them is

$$D_t(f) = \min_{\substack{x \in \mathcal{S}_t \\ (x,f) \in E}} \{d(x) + w(x, f)\}.$$

All longer candidates to this same endpoint can now be excluded; if candidates tie, one representative suffices. Thus Step 2 comprises both optimal-prefix replacement and same-endpoint minimization. Both reductions are reflected in the relaxation operation. After Step 2, the current search is represented by the frontier values

$$\{D_t(f) : f \in \mathcal{F}_t\}.$$

4.3 Step 3: Global minimum extraction

Choose

$$f^* \in \arg\min_{f \in \mathcal{F}_t} D_t(f), \qquad D^* = D_t(f^*).$$

We show that this is the minimum not only among frontier representatives but among *all* paths in $\mathcal{P}(\mathcal{U}_t)$.

Take any $P \in \mathcal{P}(\mathcal{U}_t)$. Let f be its first unsolved vertex, x its predecessor, and L_x the length of the prefix from s to x. As $x \in \mathcal{S}_t$, $d(x) \le L_x$. Since $D_t(f)$ is the least candidate over all solved-to-f boundary edges,

$$D_t(f) \le d(x) + w(x, f) \le L_x + w(x, f).$$

The remainder of P after f has nonnegative length, so $w(P) \geq L_x + w(x, f)$. By the choice of f^*, $D_t(f) \geq D_t(f^*)$. Combining gives

$$\begin{aligned} w(P) &\geq L_x + w(x, f) \\ &\geq d(x) + w(x, f) \\ &\geq D_t(f) \\ &\geq D_t(f^*) = D^*. \end{aligned}$$

A path attaining D^* exists, namely an already finalized shortest prefix followed by the corresponding boundary edge. Therefore

$$D^* = \min_{P \in \mathcal{P}(\mathcal{U}_t)} w(P).$$

Theorem 1 (Correctness of one iteration)**.** *If all edge weights are positive and at least one unsolved vertex is reachable, then any f^* minimizing $D_t(f)$ over $\mathcal{F}_t$ satisfies*

$$D_t(f^*) = \delta(s, f^*).$$

Hence its shortest-path distance can be finalized.

Proof. By the preceding inequalities, a path attaining $D_t(f^*)$ is globally shortest in $\mathcal{P}(\mathcal{U}_t)$. In particular, it is shortest among the paths to its own endpoint f^*, which form a subset of $\mathcal{P}(\mathcal{U}_t)$. □

The solved state then grows and the unsolved endpoint set shrinks:

$$\mathcal{S}_{t+1} = \mathcal{S}_t \cup \{f^*\}, \qquad \mathcal{U}_{t+1} = \mathcal{U}_t \setminus \{f^*\}.$$

The same reasoning applies at the next iteration.

4.4 Correspondence with the standard algorithm

The three exclusions are not additional operations. They account for the operations already performed by the standard algorithm, as summarized in Table 1. Frontier restriction implements Step 1; relaxation maintains the reduced candidate values of Step 2; and Extract-Min implements Step 3.

Table 1: Exclusion steps and corresponding operations in Dijkstra's algorithm.

Exclusion step	Standard operation	Mathematical role
Step 1: Frontier exclusion	Frontier restriction	Omit paths that continue beyond their first unsolved vertex.
Step 2: Relaxation exclusion	Relaxation of tentative distances	Use optimal solved prefixes and retain the best candidate for each frontier endpoint.
Step 3: Global minimum	Extract-Min	Select the global minimum among the surviving candidates.

To make the correspondence between the three-step exclusion proof and the original algorithm more explicit, Algorithm 1 presents a compact version of Dijkstra's procedure, with the relevant operations annotated. It starts with the source s in the solved set and immediately relaxes its outgoing edges. The first minimum extraction therefore uses frontier values that have already

been computed. In later iterations, the relaxations following an extraction update the frontier values for the next extraction. Hence, although the loop places EXTRACT-MIN before relaxation, the state used by each extraction already reflects Step 2. Step 1 is implicit in that state: an unsolved vertex not yet exposed by the solved set has distance label ∞, so it cannot be chosen while a reachable frontier candidate remains.

Algorithm 1 Dijkstra's algorithm with the source settled initially

```
1: d(s) ← 0; d(v) ← ∞ for v ≠ s
2: S ← {s}; Q ← V \ {s} (queue keyed by d)
3: for each (s, v) ∈ E with v ∈ Q do
4:     d(v) ← min{d(v), w(s, v)}; update v's key in Q          ▷ Step 2: initial relaxation
5: end for
6: while Q ≠ ∅ do
7:     u ← ExtractMin(Q)                                      ▷ Step 3: frontier minimum
8:     if d(u) = ∞ then
9:         break
10:    end if
11:    S ← S ∪ {u}
12:    for each (u, v) ∈ E with v ∈ Q do
13:        if d(u) + w(u, v) < d(v) then
14:            d(v) ← d(u) + w(u, v); update v's key in Q     ▷ Step 2: relaxation
15:        end if
16:    end for
17: end while
```

Dijkstra's original account already uses three vertex classes and a sequence of candidate updates followed by minimum selection [1]. The present proof identifies the global path-minimization argument behind that established process. It is a proof of the original algorithm, not a modified algorithm.

5 Discussion: Dynamic Programming Rather Than Greedy Selection

5.1 The dynamic-programming recurrence

The recursive structure follows directly from Step 2. For each frontier vertex f,

$$D_t(f) = \min_{\substack{x \in \mathcal{S}_t \\ (x,f) \in E}} \{d(x) + w(x, f)\}.$$

All distances $d(x)$ on the right are final. The global frontier minimum establishes one more final distance, which is then available in later iterations. In this way, the set of solved shortest paths grows through repeated applications of optimal substructure. The finalized distances form a nondecreasing sequence, as successive iterations minimize over progressively smaller sets of unsolved destinations.

The recurrence can be compared with the familiar Fibonacci computation,

$$F_0 = 0, \qquad F_1 = 1, \qquad F_n = F_{n-1} + F_{n-2}.$$

Both procedures construct a new exact result from previously solved subproblems. Their dependency structures differ. The next Fibonacci index is fixed in advance. In Dijkstra, the next

vertex is determined by the current frontier and its weights. Nevertheless, neither computation requires guessing the value of the next solved subproblem. This common incremental structure supports the dynamic-programming interpretation of Dijkstra.

5.2 Why the greedy classification is incorrect

The standard greedy account emphasizes the immediate minimum-distance selection. The usual meaning of this account is a decision based on the best available local criterion, without first examining all possible future continuations. Kleinberg and Tardos explicitly describe the stepwise rule of a greedy algorithm as *myopic* [4]. Such a description may suggest an optimistic local choice whose success has to be established afterward.

The textbook correctness argument commonly follows that order: select the minimum tentative label, then prove that its value is final for the selected vertex. These proofs are mathematically valid, and they use comparisons with paths crossing the settled boundary. Yet their stated target is the optimality of that particular endpoint. They do not begin by minimizing over the union of *all* paths ending at unsolved vertices. The global-to-local proof makes that stronger minimization the starting point of the argument.

Our three exclusions establish, before finalizing the next vertex, that no path in the current global space is shorter than the chosen boundary candidate. The minimum is therefore not an optimistic, short-sighted, or blind local guess. It is a consequence of a complete and directed elimination of inferior paths. Moreover, the procedure relies on shortest paths already obtained and uses them to determine a new shortest path. We accordingly reject the greedy classification and identify dynamic programming as the more accurate account of Dijkstra's algorithm. Its minimum-extraction operation is the final expression of deterministic global exclusion, not the origin of the algorithm's correctness.

6 Conclusion

This paper has given a global-to-local proof of Dijkstra's algorithm. Three deterministic exclusions reduce the paths to unsolved destinations to frontier candidates, then to one optimal candidate per frontier vertex, and finally to the global shortest path. Its endpoint's shortest-path optimality follows directly from that global minimum.

Each iteration uses previously established shortest paths to determine one new shortest path. This is a dynamic-programming process, not a speculative local greedy choice. The result places Dijkstra alongside Bellman–Ford and Floyd–Warshall, which are already widely understood as dynamic-programming shortest-path algorithms [5]. It thus provides a more unified conceptual classification of these major classical shortest-path algorithms.

Acknowledgments

The author thanks Zimo Wang for helpful discussions that made the central ideas of this work clearer. The author also acknowledges the assistance of AI large language models, specifically Gemini and ChatGPT, in drafting, translation, and editorial refinement of the manuscript.

References

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959. doi:10.1007/BF01386390.

[2] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ: Addison-Wesley Professional, 2011.

[3] G. Brassard and P. Bratley, *Algorithmics: Theory and Practice*. Englewood Cliffs, NJ: Prentice Hall, 1988.

[4] J. Kleinberg and E. Tardos, *Algorithm Design*. Boston, MA: Pearson/Addison-Wesley, 2006.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.

[6] K. Wayne, "COS 423: Analysis of Algorithms—Greedy Algorithms: Shortest Path," Princeton University, course materials. [Online]. Available: https://www.cs.princeton.edu/~wayne/cs423/.

[7] T. Roughgarden, *Algorithms Illuminated, Part 2: Graph Algorithms and Data Structures*. San Francisco, CA: Soundlikeyourself Publishing, LLC, 2018.

[8] R. Bellman, *Dynamic Programming*. Princeton, NJ: Princeton University Press, 1957.

[9] E. L. Lawler, *Combinatorial Optimization: Networks and Matroids*. New York, NY: Holt, Rinehart and Winston, 1976.

[10] M. Sniedovich, "Dijkstra's algorithm revisited: The dynamic programming connexion," *Control and Cybernetics*, vol. 35, no. 3, pp. 599–620, 2006.

[11] J. Misra, "A walk over the shortest path: Dijkstra's algorithm viewed as fixed-point computation," *Information Processing Letters*, vol. 77, nos. 2–4, pp. 197–200, 2001. doi:10.1016/S0020-0190(00)00202-7.

[12] A. Lew, "Canonical greedy algorithms and dynamic programming," *Control and Cybernetics*, vol. 35, no. 3, pp. 621–643, 2006.